

React Practice Programs

Component Props, Children, Form Handling & Conditional Rendering

Program 1

Profile and Gallery Components (Default Export & Named Export)

Create a Profile component (default export) that renders a person's name, image, and description. Create a Gallery component (named export) that imports Profile and renders it. Render the Gallery component inside App.

Profile.js

```
import './styles.css';

export default function Profile() {
  return (
    <div className="profile-card">
      <h2>Marie Curie</h2>
      
      <p>
        Marie Curie was a physicist and chemist known for her pioneering
        research on radioactivity. She was the first person to win Nobel
        Prizes in two different sciences.
      </p>
    </div>
  );
}
```

Gallery.js

```
import Profile from './Profile';

export function Gallery() {
  return (
    <section className="gallery">
      <h1>Notable Scientists</h1>
      <Profile />
    </section>
  );
}
```

App.js

```
import { Gallery } from './Gallery';

export default function App() {
  return (
    <div className="app">
      <Gallery />
    </div>
  );
}
```

styles.css

```
.app {
  font-family: sans-serif;
  text-align: center;
  padding: 20px;
}
.profile-card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  max-width: 300px;
  margin: 16px auto;
}
.profile-image {
  border-radius: 50%;
  width: 100px;
  height: 100px;
  object-fit: cover;
}
```

Note: Profile uses 'export default', so it can be imported under any name (import Profile from './Profile'). Gallery uses a named export, so it must be imported with curly braces: import { Gallery } from './Gallery'.

Program 2

Profile and Card Components (Props & children)

Create two scientist objects (scientist, scientist2) with id, name, image, profession, and description. Create a Profile component that takes scientist details as props. Create a Card component that accesses Profile via the children prop. Render two Card components in App, each wrapping a Profile with a different scientist's data.

Profile.js

```
export default function Profile({ name, image, profession, description }) {  
  return (  
    <div className="profile">  
      <h2>{name}</h2>  
      <img src={image} alt={name} className="profile-image" />  
      <p className="profession">{profession}</p>  
      <p>{description}</p>  
    </div>  
  );  
}
```

Card.js

```
export default function Card({ children }) {  
  return <div className="card">{children}</div>;  
}
```

App.js

```

import Card from './Card';
import Profile from './Profile';
import './styles.css';

const scientist = {
  id: 1,
  name: 'Marie Curie',
  image: 'https://placeholder.co/150x150',
  profession: 'Physicist & Chemist',
  description:
    'Pioneered research on radioactivity and won Nobel Prizes in two different sciences.',
};

const scientist2 = {
  id: 2,
  name: 'Albert Einstein',
  image: 'https://placeholder.co/150x150',
  profession: 'Theoretical Physicist',
  description:
    'Developed the theory of relativity and contributed to quantum mechanics.',
};

export default function App() {
  return (
    <div className="app">
      <Card>
        <Profile {...scientist} />
      </Card>
      <Card>
        <Profile {...scientist2} />
      </Card>
    </div>
  );
}

```

styles.css

```

.app {
  display: flex;
  gap: 20px;
  justify-content: center;
  padding: 20px;
  font-family: sans-serif;
}

.card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  width: 220px;
  text-align: center;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.profile-image {
  border-radius: 50%;
  width: 100px;
  height: 100px;
  object-fit: cover;
}

.profession {
  font-style: italic;
  color: #666;
}

```

Note: Card does not know what is inside it ahead of time - it renders whatever is passed between its opening and closing tags as the children prop. Here that is a Profile component with the scientist data spread onto it as props.

Program 3

Login Component with Form Validation

Develop a Login component with Username and Password fields and a Login button. Implement form validation: if the user submits without entering a username or password, display an error for empty fields. On successful completion, acknowledge the login as successful.

Login.js

```
import { useState } from 'react';

export default function Login() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(false);

  function handleSubmit(e) {
    e.preventDefault();

    if (!username || !password) {
      setError('Please enter both username and password.');
```

```
      setSuccess(false);
      return;
    }

    setError('');
    setSuccess(true);
  }

  return (
    <form className="login-form" onSubmit={handleSubmit}>
      <label>Username:</label>
      <input
        type="text"
        placeholder="Enter Name"
        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />

      <label>Password:</label>
      <input
        type="password"
        placeholder="Enter password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />

      <button type="submit">Login</button>

      {error && <p className="error">{error}</p>}
      {success && <p className="success">Login successful!</p>}
    </form>
  );
}
```

App.js

```
import Login from './Login';
import './styles.css';

export default function App() {
  return (
    <div className="app">
      <Login />
    </div>
  );
}
```

styles.css

```
.app {
  font-family: sans-serif;
  padding: 30px;
}
.login-form {
  display: flex;
  flex-direction: column;
  max-width: 320px;
}
.login-form label {
  margin-top: 10px;
  margin-bottom: 4px;
  font-size: 14px;
}
.login-form input {
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.login-form button {
  margin-top: 16px;
  padding: 8px;
  background-color: #1877f2;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 80px;
}
.error {
  color: red;
  margin-top: 10px;
}
.success {
  color: green;
  margin-top: 10px;
}
```

Note: The inputs are controlled (value + onChange tied to state), so username and password always reflect what is typed. On submit, handleSubmit checks if either field is empty and sets an error message; otherwise it clears the error and shows a success message.

Program 4

Product List with Conditional Rendering

Create a `productsData` array of product objects (`id`, `name`, `price`, `description`, `available`). Create a `ProductList` component that maps through `productsData` and renders a `Product` component for each item. Create a `Product` component that conditionally renders `name/price/description` when `available` is `true`, or an 'Out of stock' message when `available` is `false`. Render `ProductList` inside `App`.

productsData.js

```
export const productsData = [
  {
    id: 1,
    name: 'Wireless Mouse',
    price: 799,
    description: 'A smooth and responsive wireless mouse.',
    available: true,
  },
  {
    id: 2,
    name: 'Mechanical Keyboard',
    price: 2499,
    description: 'RGB backlit mechanical keyboard with blue switches.',
    available: false,
  },
  {
    id: 3,
    name: 'USB-C Hub',
    price: 1299,
    description: 'Multi-port hub with HDMI, USB 3.0, and SD card slot.',
    available: true,
  },
];
```

Product.js

```
export default function Product({ product }) {
  const { name, price, description, available } = product;

  return (
    <div className="product-card">
      <h3>{name}</h3>
      {available ? (
        <>
          <p className="price">Rs. {price}</p>
          <p>{description}</p>
        </>
      ) : (
        <p className="out-of-stock">Out of stock</p>
      )}
    </div>
  );
}
```

ProductList.js


```
import Product from './Product';
import { productsData } from './productsData';

export default function ProductList() {
  return (
    <div className="product-list">
      {productsData.map((product) => (
        <Product key={product.id} product={product} />
      ))}
    </div>
  );
}
```

App.js

```
import ProductList from './ProductList';
import './styles.css';

export default function App() {
  return (
    <div className="app">
      <h1>Our Products</h1>
      <ProductList />
    </div>
  );
}
```

styles.css

```
.app {
  font-family: sans-serif;
  text-align: center;
  padding: 20px;
}
.product-list {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
  justify-content: center;
}
.product-card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  width: 200px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}
.price {
  font-weight: bold;
  color: #1877f2;
}
.out-of-stock {
  color: red;
  font-style: italic;
}
```

Note: Product destructures name, price, description, and available from the product prop, then a ternary decides which block to render. ProductList loops over productsData with map() and renders one Product per item, passing the whole object as a single product prop and using id as the React key.